Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Программирование графических процессоров

Лабораторная работа №7

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2024

Оглавление

Задание: 3
Цель: 3
Ход работы: 4
Вывод: 5
Листинг: 6
main.cu 6

Задание:

- Реализовать вычисление скалярного произведения векторов на GPU, используя CUDA API (“сырой код”) и, отдельно, используя библиотеку Thrust. Сравнить время выполнения программ при различной длине векторов.
- Реализовать транспонирование матрицы на GPU, используя CUDA API (“сырой код”) и, отдельно, используя библиотеку Thrust. Сравнить время выполнения программ при различной размерности матрицы.

Цель: освоить использование библиотеки Thrust.

Ход работы:

1. Реализуем вычисления скалярного произведения векторов на GPU, используя CUDA API – и, отдельно, используя библиотеку Thrust (код программы представлен в листинге).
2. Реализуем транспонирование матрицы на GPU, используя CUDA API и, отдельно, используя библиотеку Thrust (код программы представлен в листинге).

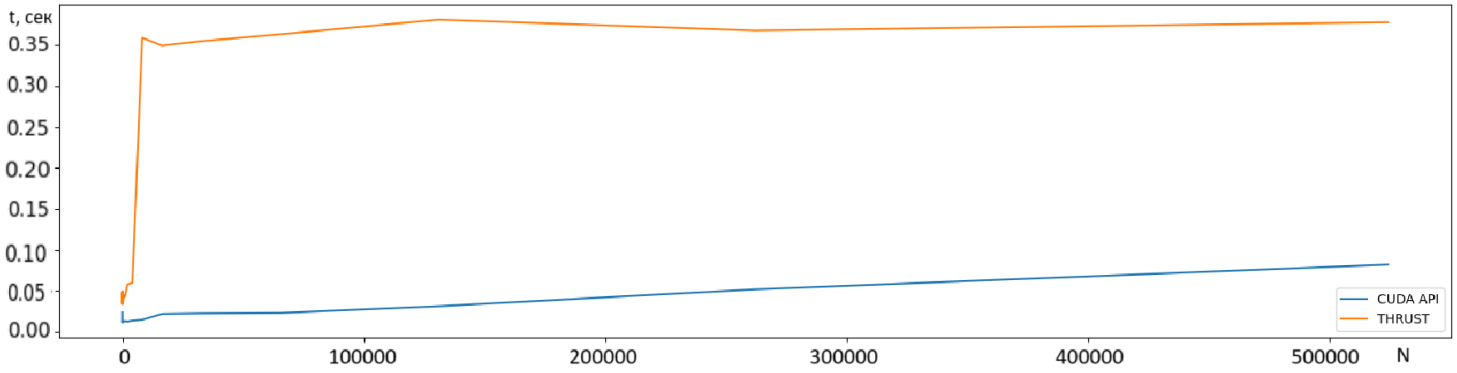


Рис. 1 – Зависимость времени вычисления скалярного произведения векторов от их размера (синий – CUDA API; оранжевый – Thrust).

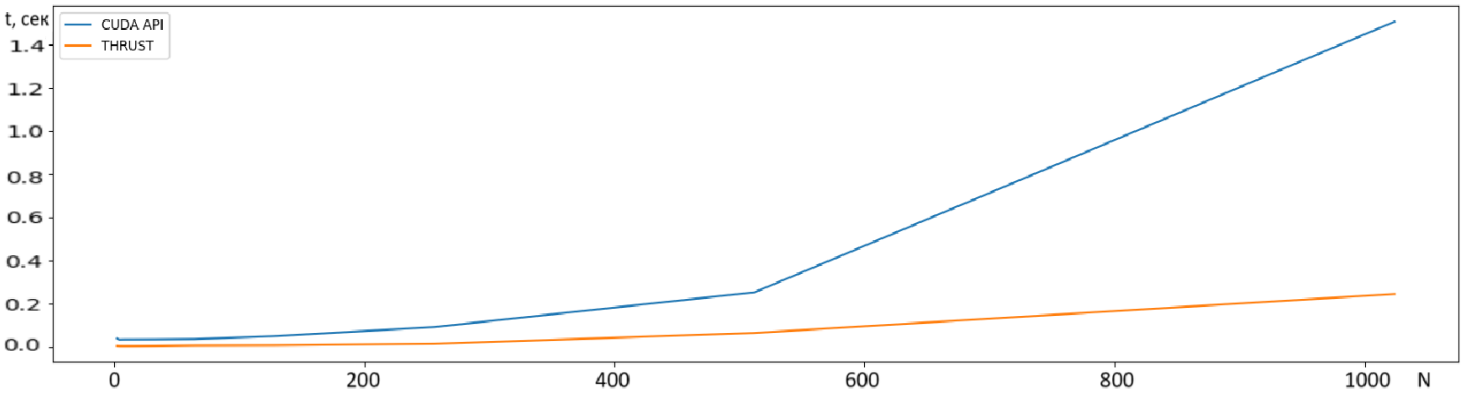


Рис. 2 – Зависимость времени выполнения транспонирования матрицы от её размерности (синий – CUDA API; оранжевый – Thrust).

Как можно увидеть из результатов:

- Вычислении скалярного произведения векторов при любых размерах векторов быстрее выполняется при использоваии CUDA API.
- Транспонирование матрицы во всх случаях быстрее выполняется при изпользовании библиотеки Thrust.

Вывод:

- Мы реализовали вычисление скалярного произведения векторов на GPU, используя CUDA API (“сырой код”) и, отдельно, используя библиотеку Thrust.
- Реализовали транспонирование матрицы на GPU, используя CUDA API (“сырой код”) и, отдельно, используя библиотеку Thrust.
- Сравнили время выполнения программ:

  Вычислении скалярного произведения векторов при любых размерах векторов быстрее выполняется при использоваии CUDA API.

  Транспонирование матрицы во всх случаях быстрее выполняется при изпользовании библиотеки Thrust.
- Освоили использование библиотеки Thrust.

Листинг:

main.cu

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>
#include <thrust/device_vector.h>
#include <thrust/inner_product.h>
#include <time.h>
#include <iostream>
#include <fstream>
#include <thrust/gather.h>

#define THREADS_PER_BLOCK 32

__global__ void dot(int *a, int *b, int *c)
{
    __shared__ int temp[THREADS_PER_BLOCK];
    int index = threadIdx.x + blockIdx.x * blockDim.x;
    temp[threadIdx.x] = a[index] * b[index];

    __syncthreads();

    if (threadIdx.x == 0)
    {
        int sum = 0;
        for (int i = 0; i < THREADS_PER_BLOCK; i++)
        {
            sum += temp[i];
        }
        atomicAdd(c, sum);
    }
}

__global__ void transpose(int *in, int *out, int N)
{
    __shared__ int tile[THREADS_PER_BLOCK][THREADS_PER_BLOCK + 1];
    int x = blockIdx.x * THREADS_PER_BLOCK + threadIdx.x;
    int y = blockIdx.y * 512 + threadIdx.y;

    for (int j = 0; j < THREADS_PER_BLOCK; j += blockDim.y)
    {
        if (x < N && y + j < N)
        {
            int index_in = (y + j) * N + x;
            tile[threadIdx.y + j][threadIdx.x] = in[index_in];
        }
    }

    __syncthreads();

    x = blockIdx.y * THREADS_PER_BLOCK + threadIdx.x;
```

```
    y = blockIdx.x * THREADS_PER_BLOCK + threadIdx.y;
    for (int j = 0; j < THREADS_PER_BLOCK; j += blockDim.y)
    {
        if (x < N && y + j < N)
        {
            int index_out = (y + j) * N + x;
            out[index_out] = tile[threadIdx.x][threadIdx.y + j];
        }
    }
}

__host__ std::vector<float> compare_dot_product(int N)
{
    int *a, *b, *c;
    int *dev_a, *dev_b, *dev_c;
    int size = N * sizeof(int);
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    std::vector<float> statistics{(float)N};

    // allocate space for the variables on the device
    cudaMalloc((void **)&dev_a, size);
    cudaMalloc((void **)&dev_b, size);
    cudaMalloc((void **)&dev_c, sizeof(int));

    // allocate space for the variables on the host
    a = (int *)malloc(size);
    b = (int *)malloc(size);
    c = (int *)malloc(sizeof(int));

    // this is our ground truth
    // int sumTest = N;
    // generate numbers
    for (int i = 0; i < N; i++)
    {
        a[i] = 1;
        b[i] = 1;
    }

    *c = 0;

    cudaMemcpy(dev_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(dev_b, b, size, cudaMemcpyHostToDevice);
    cudaMemcpy(dev_c, c, sizeof(int), cudaMemcpyHostToDevice);

    cudaEventRecord(start);

    dot<<<(N / THREADS_PER_BLOCK - 1) + THREADS_PER_BLOCK,
THREADS_PER_BLOCK>>>(dev_a, dev_b, dev_c);
```

```
    cudaMemcpy(c, dev_c, sizeof(int), cudaMemcpyDeviceToHost);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    statistics.push_back(milliseconds);

    // printf("CUDA API\n");
    // printf("Time: %f\n", milliseconds);
    // printf("Are sums equal: %d\n\n", sumTest == *c);

    thrust::device_vector<int> a_thrust(N, 1), b_thrust(N, 1);

    for (int i = 0; i < N; i++)
    {
        a_thrust[i] = 1;
        b_thrust[i] = 1;
    }

    cudaEventRecord(start);

    float result = thrust::inner_product(a_thrust.begin(), a_thrust.end(),
b_thrust.begin(), 0.0f);

    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaEventElapsedTime(&milliseconds, start, stop);

    statistics.push_back(milliseconds);

    // printf("THRUST \n");
    // printf("Time: %f\n", milliseconds);
    // printf("Are sums equal: %d\n", sumTest == result);

    free(a);
    free(b);
    free(c);

    cudaFree(dev_a);
    cudaFree(dev_b);
    cudaFree(dev_c);
    // cudaFree(c);

    return statistics;
}

__host__ std::vector<float> compare_transpose(int N)
{
```

```
    int *a, *b;
    int *dev_a, *dev_b;
    int size = N * N * sizeof(int);
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    std::vector<float> statistics{(float)N};

    cudaMalloc((void **)&dev_a, size);
    cudaMalloc((void **)&dev_b, size);

    a = (int *)malloc(size);

    for (int i = 0; i < N * N; i++)
    {
        a[i] = i;
    }
    b = (int *)malloc(size);

    cudaMemcpy(dev_a, a, size, cudaMemcpyHostToDevice);

    cudaEventRecord(start);

    transpose<<<(N / THREADS_PER_BLOCK - 1) + 512, THREADS_PER_BLOCK>>>(dev_a,
dev_b, N);

    cudaMemcpy(b, dev_b, size, cudaMemcpyDeviceToHost);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    statistics.push_back(milliseconds);

    thrust::host_vector<int> h_A(N * N);
    thrust::host_vector<int> h_AT(N * N);
    thrust::device_vector<int> d_A(N * N);
    thrust::device_vector<int> d_AT(N * N);

    for (int i = 0; i < N * N; i++)
    {
        h_A[i] = i;
    }

    int map[N * N];
    for (int i = 0; i < N * N; i++)
    {
        map[i] = (i % N) * N + (i / N);
    }
    thrust::device_vector<int> d_map(map, map + N * N);
```

```
    cudaEventRecord(start);

    thrust::gather(d_map.begin(), d_map.end(), d_A.begin(), d_AT.begin());

    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
    cudaEventElapsedTime(&milliseconds, start, stop);

    statistics.push_back(milliseconds);

    // printf("THRUST \n");
    // printf("Time: %f\n", milliseconds);
    // printf("Are sums equal: %d\n", sumTest == result);

    free(a);
    free(b);

    cudaFree(dev_a);
    cudaFree(dev_b);
    // cudaFree(c);

    return statistics;
}

template <typename T>
void write_to_file(std::vector<std::vector<T>> data, std::string filename)
{
    std::ofstream out(filename);
    for (auto row : data)
    {
        std::string row_out = "";
        for (auto item : row)
        {
            row_out += std::to_string(item) + ";";
        }
        row_out.pop_back();
        out << row_out << std::endl;
    }
}

int main()
{
    std::vector<std::vector<float>> stats;
    for (int i = 1; i < 1024 * 1024; i *= 2)
    {
        stats.push_back(compare_dot_product(i));
    }
    write_to_file(stats, "dot_product.csv");
    stats.clear();
    for (int i = 2; i <= 1024; i *= 2)
```

```
    {
        stats.push_back(compare_transpose(i));
    }
    write_to_file(stats, "transpose.csv");

    return 0;
}
```